03.04.26 Dersi

STL, Collection

ADT-s (abstract data types) } C'deki stack, array, queue yapısı her yerde ortak.

↳ array-based
pointer-based } bir stack oluşturucaz. array veya ptr olabilir.

* array based'te
[i] → O(1) → problem aradan elemanı çıkarmak yok. remove işlemi yok. X
↳ remove problemi veya izin verdiğimiz tonan shift yapıcaz veya derivative (memory kullanılıyor, kaldırılan değer yerine "-1" koyuluyor)
* Shift → O(n) → Dizideki tüm elemanları sola kaydırdığı için.
* amortized cost → arrayin bir size'ı var. Array dinamik olduğundan bir bedel ödeniyor. x2, x1 olabilir. Diziyi büyütmek için tekrar size'a karar vermek gerekiyor.

dinamic, fixed sized

x2
x1.2 } sizeler ne olacak.
x5

size → initial value
bunların seçilmesi bir problem, nasıl seçilecek.

* pointer-based
↳ dizideki remove problemine çözüm oluyor.
Burada ki problem → index yapısı olmadığı için karmaşıklık artıyor.
· memory (pointer)

Burada bazı durumlarda insert ve search'te O(n) var.

* hash = map
* pythondaki yapı ptr array → her yapı

* Bu O(n)'i bir tık aşağı aldığımızda O(logn) olur. Bunun için de ağaç yapılar kullanılıyor. Bir kural uygulayarak e performans arttıran yapılar) O(n)'den O(logn)'e indiriyoruz. Bunu da O(1) yapabilmek için hash fonksiyonu kullanılır. Dizide hash kullanma gibi... Hashin problemi çakışma olabilir.

O(n) } array-based / pointer-based

↓

O(logn) → trees

↓

O(1) → hash

Heap yapısında insert ve get var. Ağaç yapısını kullanan bir kuyruğa benziyor.

* Liste için fonksiyonlar ve karmaşıklıkları:

List<T> O(n) ise ekleme yaptığın yerde sağa doğru shift ihtiyacı.

↳ . add → O(1) ise listenin sonuna ekler.
. search → O(n)
. remove → O(n)
. update → O(1), # search (O(n)) önce search kullanacağımız için O(n)

altteki yapının ptr veya array olmasına göre değişen karmaşıklıklar.

↳ . add → Buna karşılık gelen işlem (fonksiyon karşılığı aslında:
. items[count] = item;

↳ . insert(index i, T item) → [2|5|6|3|...] O(n)
i: 2, item: 9
9 6 7
2. indexe 9 ekleyelim
en sona gittiği için O(n)

Bu kısım gpt'nin yaptığı insert fonk. kodunun karmaşıklığı

→ diziye bunlar kolay işlemler.

Stack<T>

push() → O(1)

pop() → O(1)

} ekleme işlemi array-based olduğunda problemli. Size'ı geçince problem oluyor.

↳ nette otomatik olarak dizi x2 katına çıkarılır.

Stack yapısı nette array kullanıyor.

array yapısındaki kuyruk

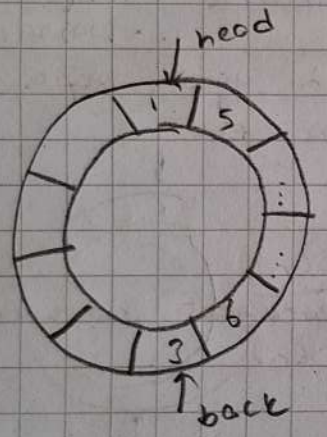
Queue<T>

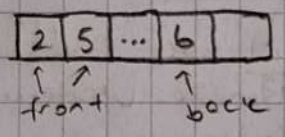
enqueue (item) → O(1)

bir array var ve bunda baş sıfırsak.

dequeue → O(1) ise shift yapıyorsa O(n) olur. Bunun yerine 2 tane ptr ile bunu yönetebiliriz. O zaman karmaşıklık O(1) olur.

en sondaki en baş çıkıyor.





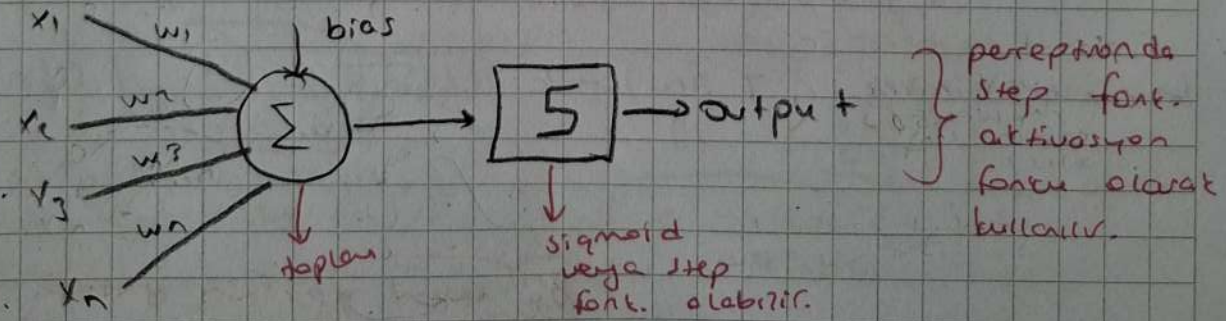
(front'u sadece kaydırdık shift yapmadık O(1) oldu. 2 ptr kullandığımız için)

sanki böylesinin gibi bir yapı.

↳ kuyrukta 1 eleman silinse size -- oldu.

* head = tail ise dizi dolmuş demektir.

* YSA'nın en temeli perceptron / primitive yapı.



} perceptron da step fonk. aktivasyon fonksiyonu olarak kullanılır.

* perceptronun karmaşıklığı
↳ algoritma → $x_i w_i$ yi çarpıyor.
tek birinin maliyeti 1.

Bizim kullandığımız x kaç tane ise o kadar çarpma işlemi var.

$x_i w_i \rightarrow x_1 w_1 + x_2 w_2 + \dots + x_n w_n$ → dark

↳ $x^T w$ → bu değeri m ile çarpıyoruz.
m → sample sayısı

MNIST'teki
m = 70000
x_i : 28 x 28 = 786

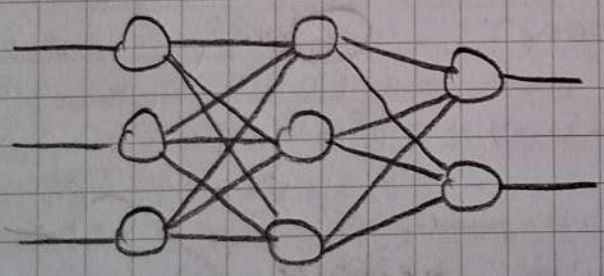
Tek bir epoch için tek bir çarpmayı 70 bin defa yaptı.

↳ complexity:
epoch x m x 786
kaç defa döngüde kaldık — feature sayısı

Burada her bir m'de 786 defa dönüyor. girdi x.

* perceptron'da bir döngüde input kadar iş yapıyor.
- Tek bir matris işlemi yapıyor. n vektörü çarpıp topluyor.
Bu algoritma perceptron veya logistic regresyon algosu.

* perceptronun bir serisi multi layer perceptron:



m feature işlemi kaç layer varsa o kadar artıyor. MNIST'teki gibi.

CNN Model

* Her bir hücreyi elde et. → Her bir hücre / feature için pooling (mask gibi)

m.w değeri geliyor.
↓
w değerine bağlı olarak maskin değerine göre çıktı gelir.

layer eklemek demek maliyeti x.w_i kadar arttırmak demektir.

Random arrayden max. bulmanın karmaşıklığı
↳ en küçüğü, en büyüğü bulmanın maliyeti O(n)
Bir grup sayının, arrayin ortalamasının karmaşıklığı
↳ toplam /n → O(n)'dir.
Dizinin std'sini bulmanın maliyeti O(n)'dir.
↳ ortalamaya göre O da O(n) ama std'yi bulmak daha problemli çünkü std'de ortalama bulunur.

Örnek Sorular (Deste Bakılı)

- Bir array bubble ile 5 ms sıralanıyor. Bu array sizeını x4 yaparsak bubble sorte sıralama zaman beklentimiz ne olur?

karmaşıklık → $O(n^2)$ olduğundan:

size → x4 olursa zaman $4^2 = 16$ kat artar.

Yani 5ms x16 = 80 ms

- NP, P problemler

rekursive fibo np bir problem

40'a karşılık gelen 1 saat ise 50'e kaç saatte bulunur.

Rekursive Fibo karmaşıklığı $O(2^n)$ olup

$n = 40 \rightarrow 1$ saat

$n \rightarrow 50 \rightarrow 1024$ saat.

$$\begin{array}{ccc} 2^{40} & \times & 1 \text{ saat} \\ 2^{50} & & x \end{array}$$

$\rightarrow x = \frac{2^{50}}{2^{40}}$

$\Rightarrow x = 2^{10}$

$n \rightarrow 50 \rightarrow 2^{(50-40)} = 2^{10} = 1024$

Cayın algo yat 40'a 1 saat kuran np esasını 100 değerini kaç yılda hesaplar.

$n \rightarrow 40 \rightarrow 1$ saat

$n \rightarrow 100 \rightarrow 2^{(100-40)} = 2^{60}$ saat daha uzun

$\underbrace{1{,}31 \text{ trilyon yıl}}_{2^{60} \text{ saatin yıl karşılığı.}} \rightarrow 1.31 \times 10^{14}$ yıl